

DS 203 – Big Data Essentials

Advanced PostgreSQL – Views

Setting up the database:

For today's session we will be using the sample *dvdrental* database. All the information about the database and instructions to download it are accessible through the following link:

<https://neon.com/postgresql/postgresql-getting-started/postgresql-sample-database>

After downloading the database, **you need to import it** into your PostgreSQL using command line (psql and pg_restore tool) or pgAdmin. Use the following link to follow the instructions on how to import/load the data:

<https://neon.com/postgresql/postgresql-getting-started/load-postgresql-sample-database>

PostgreSQL Views & Materialized Views

Why Use Views?

A view is a saved SQL query that acts like a virtual table. It's not stored with data, but can be used just like a table in queries.

Benefits of Using Views:

- **Simplicity:** Hide complex queries behind a simple interface.
- **Security:** Show only specific columns to users (e.g., no credit card info).
- **Reusability:** Write query logic once, reuse in multiple places.
- **Maintainability:** Update the query logic in one place.
- **Abstraction:** Decouple front-end applications from table structure changes.

SECTION 1: Working with Simple Views

1.1 Create a View

Use Case: Show a simplified list of customers and their cities.

```
CREATE VIEW customer_city_view AS
SELECT
    first_name,
    last_name,
    email,
    city
FROM
    customer
JOIN
    address USING (address_id)
JOIN
    city USING (city_id);
```

What this does:

Joins three tables: customer, address, and city

Returns selected columns as a virtual table

1.2 Query a View

```
SELECT * FROM customer_city_view WHERE city = 'London';
```

This behaves like a regular table, but the data is fetched fresh from the base tables.

1.3 Drop a View

```
DROP VIEW customer_city_view;
```

1.4 Replace or Update a View

Option 1: Drop and recreate

```
DROP VIEW IF EXISTS customer_city_view;
```

```
CREATE VIEW customer_city_view AS
```

```
SELECT
```

```
    first_name,
```

```
    last_name,
```

```
    email,
```

```
    city,
```

```
    country
```

```
FROM
```

```
    customer
```

```
JOIN
```

```
    address USING (address_id)
```

```
JOIN
```

```
    city USING (city_id)
```

```
JOIN
```

```
    country USING (country_id);
```

Option 2: Use CREATE OR REPLACE VIEW

```
CREATE OR REPLACE VIEW customer_city_view AS
```

```
SELECT
```

```
    first_name,
```

```
    last_name,
```

```
    city
```

```
FROM
```

```
    customer
```

```
JOIN
```

```
    address USING (address_id)
```

```
JOIN
```

```
    city USING (city_id);
```

SECTION 2: Materialized Views in PostgreSQL

What is a Materialized View?

A materialized view is a stored snapshot of the result of a query. Unlike normal views, it stores data on disk and must be refreshed manually.

Benefits of Materialized Views

Improved performance – especially for complex joins/aggregations

Faster reads – data is precomputed

Useful for reporting/dashboarding

Reduces database load on frequently accessed queries

2.1 Create a Materialized View

Use Case: Get the total number of rentals per customer

```
CREATE MATERIALIZED VIEW rental_summary AS
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    COUNT(r.rental_id) AS total_rentals
FROM
    customer c
JOIN
    rental r ON c.customer_id = r.customer_id
GROUP BY
    c.customer_id;
```

2.2 Query a Materialized View

```
SELECT * FROM rental_summary ORDER BY total_rentals DESC;
```

2.3 Refresh a Materialized View

Materialized view stores snapshot of data and doesn't get updated automatically. Instead, you have to refresh it manually before it shows you the data updates.

Let's see the effect of REFRESHING the Materialized View. First we will simulate a new rental to affect the underlying data and then refresh the view to see the updates.

We'll **insert a new row into the rental table**, using one of the existing inventory_ids and customer_ids. You can find them with:

```
SELECT inventory_id FROM rental LIMIT 1;
```

```
SELECT customer_id FROM customer LIMIT 1;
```

Assume you found:

- inventory_id = 1
- customer_id = 1
- staff_id = 1 (you can confirm this with `SELECT * FROM staff;`)

Now insert a new rental:

```
INSERT INTO rental (rental_date, inventory_id, customer_id, return_date, staff_id)  
VALUES (NOW(), 1, 1, NULL, 1);
```

After you insert, the materialized view will not get updated automatically.

Check the View Again (Before Refreshing)

```
SELECT * FROM rental_summary WHERE customer_id = 1;
```

You will not see the new rental reflected yet because the view still holds the old snapshot.

Refresh the Materialized View:

Use the following command to refresh the materialized view:

```
REFRESH MATERIALIZED VIEW rental_summary;
```

Now check again:

```
SELECT * FROM rental_summary WHERE customer_id = 1;
```

Now you should see the total_rentals increased by 1.

2.4 Drop a Materialized View

```
DROP MATERIALIZED VIEW rental_summary;
```

Summary: View vs Materialized View

Feature	View	Materialized View
Stores data	✗ No	✓ Yes (snapshot)
Always up to date	✓ Yes	✗ No (needs manual refresh)
Performance	Slower (live query)	Faster (precomputed)
Use case	Dynamic data, abstraction	Static reporting, performance boost
Can be indexed	✗ No	✓ Yes (just like tables)

References:

Neon. (n.d.-a). *PostgreSQL sample database*. Retrieved August 5, 2025, from <https://neon.com/postgresql/postgresql-getting-started/postgresql-sample-database>

Neon. (n.d.-b). *Load PostgreSQL sample database*. Retrieved August 5, 2025, from <https://neon.com/postgresql/postgresql-getting-started/load-postgresql-sample-database>

OpenAI. (2025). *PostgreSQL views and materialized views: A beginner's guide using the dvdrental database* [AI-generated instructional content]. ChatGPT. <https://chat.openai.com/>